

6 命令模式

封装调用

这些绝密文件投递箱已经促成了间谍工业的革命。我只要把要求丢进去，就会有人消失，政府一夜之间改朝换代，我的干洗工作也好了。我不必操心何时、何处或者如何完成，反正就是搞定了！



本章中，我们把封装带到全新的高度：我们打算封装方法调用。没错，通过封装方法调用，我们可以把小块计算包装成形，以便调用计算的对象不需要操心如何做事情，只需使用我们包装成形的方法。有了这些封装好的方法调用，我们也可以聪明地做一些事情，像记录日志，或者复用它们以实现撤销（undo）功能。



Home Automation or Bust, Inc.
1221 Industrial Avenue, Suite 2000
Future City, IL 62914

您好!

最近Johnny Hurricane (Weather-O-Rama气象站CEO) 向我展示并简单介绍了新的可扩展气象站。不得不说, 该软件的架构给我留下了深刻的印象, 因此想邀请您来为我们新的家居自动化遥控器设计API。为了回报您的服务, 我们很乐意向您提供Home Automation or Bust公司的股票期权。

附上一个创新控制器的原型供您研究。这个遥控器具有七个可编程的槽(每个都可以分配给一个不同的家电设备)以及对应的开关按钮。这个遥控器也有一个全局的撤销按钮。

我也在这个邮件中附上一组Java类, 这些类是由多家厂商开发出来的, 用来控制家电自动化设备, 例如, 灯、风扇、热水器、音响设备, 以及其他类似的可控制设备。

希望您创建给遥控器编程用的API, 让每个槽可以被分配控制一个或一组设备。请注意, 要能够控制目前所有的设备以及厂商未来可能提供的任何设备, 这一点很重要。

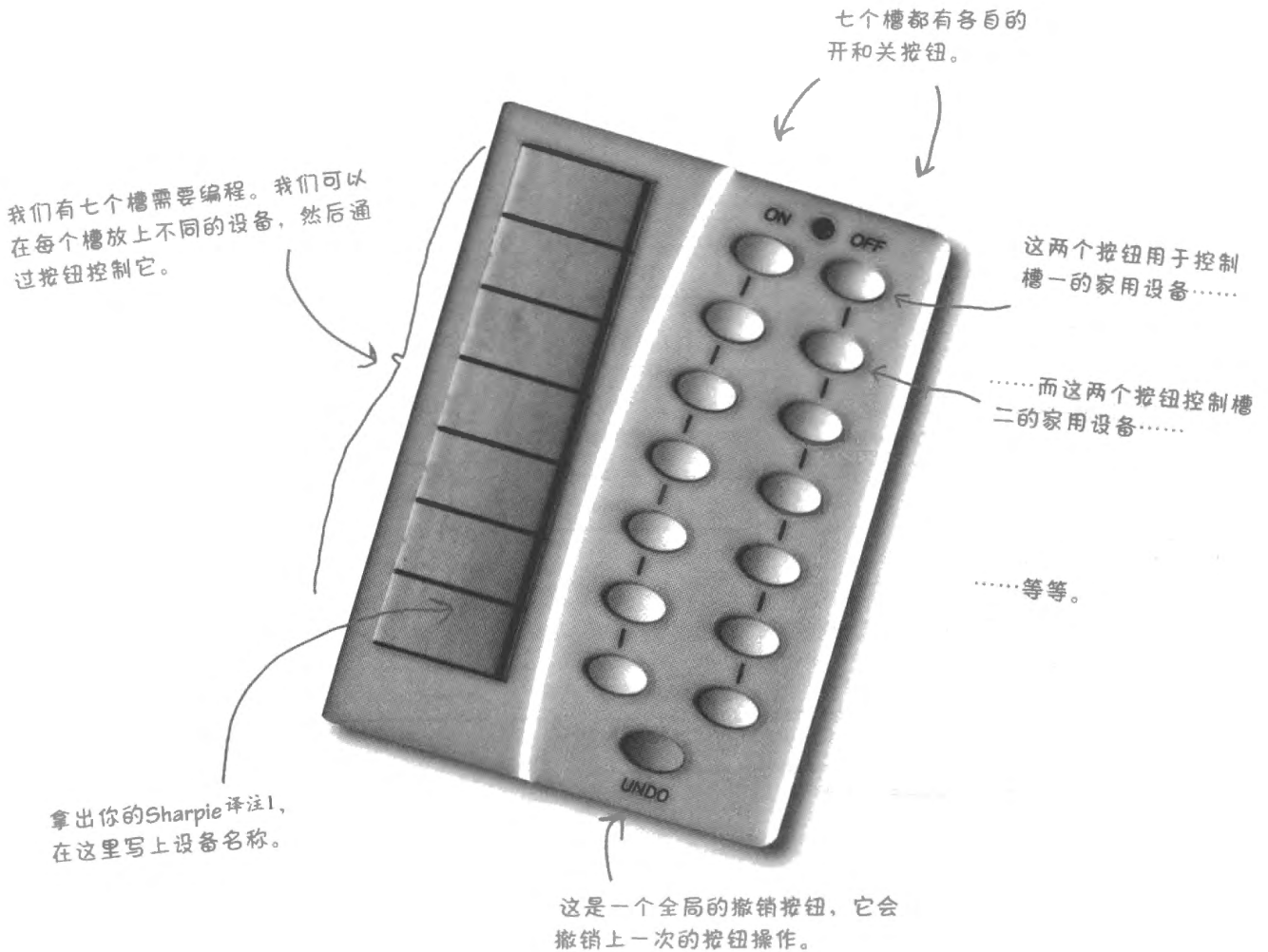
基于你在Weather-O-Rama气象站上所做的工作, 我们知道您一定会在遥控器上有出色的表现!

期待看到你的设计。

诚挚的,

Bill Thompson, CEO

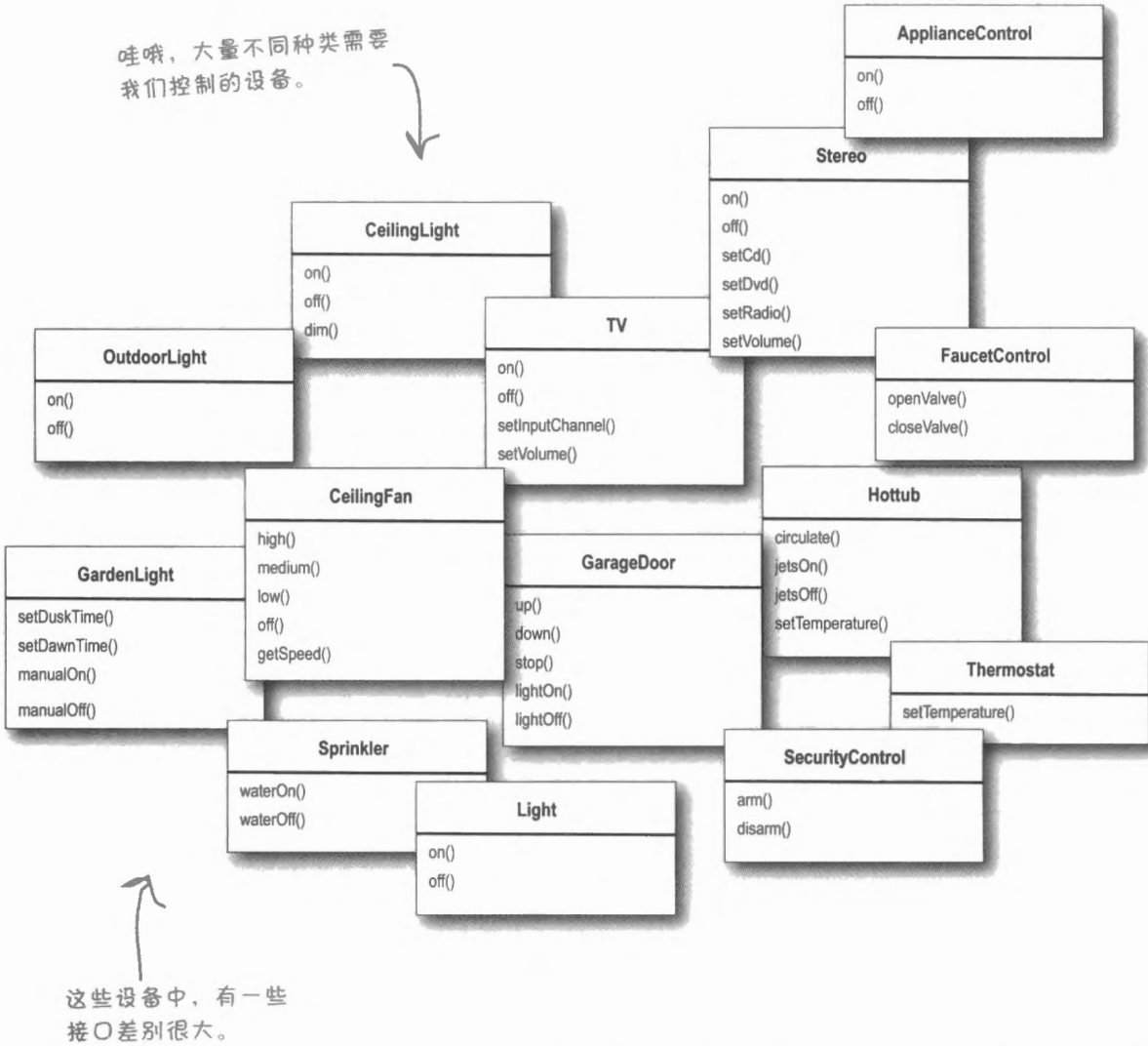
解放硬件！我们来看看遥控器……



译注1：著名的记号笔品牌。

看看厂商类

我们来看看CEO在邮件中附带的厂商类。关于我们需要从遥控器控制的对象的接口，这些类应该会给你带来一些想法。



看起来我们有很多类，而行业中在通用接口上付出的努力并不多。不仅如此，听起来将来还会有更多这样的类。设计一个遥控器API很有意思，我们继续吧。

办公室隔间对话

你的队友们已经在讨论如何设计遥控器API……



Mary: 是的，我们有一些带有on()和off()方法的类，但这些类还有其他一些方法，像dim()、setTemperature()、setVolume()、setInputChannel()和waterOn()。

Sue: 还不只这样，听起来将来可能还有更多的厂商类，方法各异。

Mary: 我认为，重要的是，我们把这看作关注的分离。

Sue: 意思是？

Mary: 我的意思是，遥控器应该知道如何解释按钮被按下的动作并做出请求，但它不应该知道关于家居自动化或如何打开热水器的太多事情。

Sue: 但是如果遥控器是“哑”的，只知道如何做通用的请求，我们怎么设计遥控器使得它可以调用开灯或开车库门等动作？

Mary: 我不确定该怎么做，但遥控器不必知道厂商类的特定内容。

Sue: 这话什么意思？

Mary: 我们不想让遥控器包含一套if语句，像“if slot1 == Light, then light.on(), else if slot1 == Hottub then hottub.jetsOn()”。我们知道这是糟糕的设计。

Sue: 我同意。无论何时有新的厂商类出现，我们不得不进入代码内部来修改，这会给我们自己带来潜在的bug以及更多的工作！



嘿，嘿！我不小心听到你们的对话。从第1章开始，我就在努力地学习设计模式。有一个模式叫做“命令模式”，我认为可能有帮助。

Mary: 是吗？再多说一些来听听。

Joe: 命令模式能让你把动作的请求者从实际执行动作的对象解耦。因此，在这里，请求者是遥控器，执行动作的对象是厂商类之一的实例。

Sue: 这怎么可能？我们怎样才能解耦它们？毕竟，当我按下按钮时，遥控器必须把灯打开。

Joe: 通过在你的设计中引入命令对象。你可以做到这一点。命令对象把做一些事情（像开灯）的请求封装成一个特定对象（例如，起居室灯对象）。因此，如果我们为每个按钮储存一个命令对象，当按钮被按下时，我们就请命令对象做一些工作。遥控器不知道工作内容是什么，只要有个命令对象知道如何和正确的对象沟通，把事情做好即可。因此你看，遥控器从灯对象解耦了！

Sue: 这无疑听起来像是一个正确的方向。

Mary: 我仍然难以理解这个模式怎么运作。

Joe: 考虑到对象如此解耦，要描述这个模式的实际运作是有点难度的。

Mary: 听听我的想法是否正确：使用这个模式，我们能够创建一个API，将这些命令对象加载到按钮槽，让遥控器的代码保持非常简单。命令对象把如何做家居自动化任务以及需要做任务的对象封装在一起。

Joe: 是的，我认为是这样。我也认为这个模式可以帮助你实现撤销按钮，但我还没有学到这部分。

Mary: 这听起来真是令人振奋，但我想，要真正理解这个模式，我还有一些工作要做。

Sue: 我也是。

与此同时，回到餐馆……

或者，

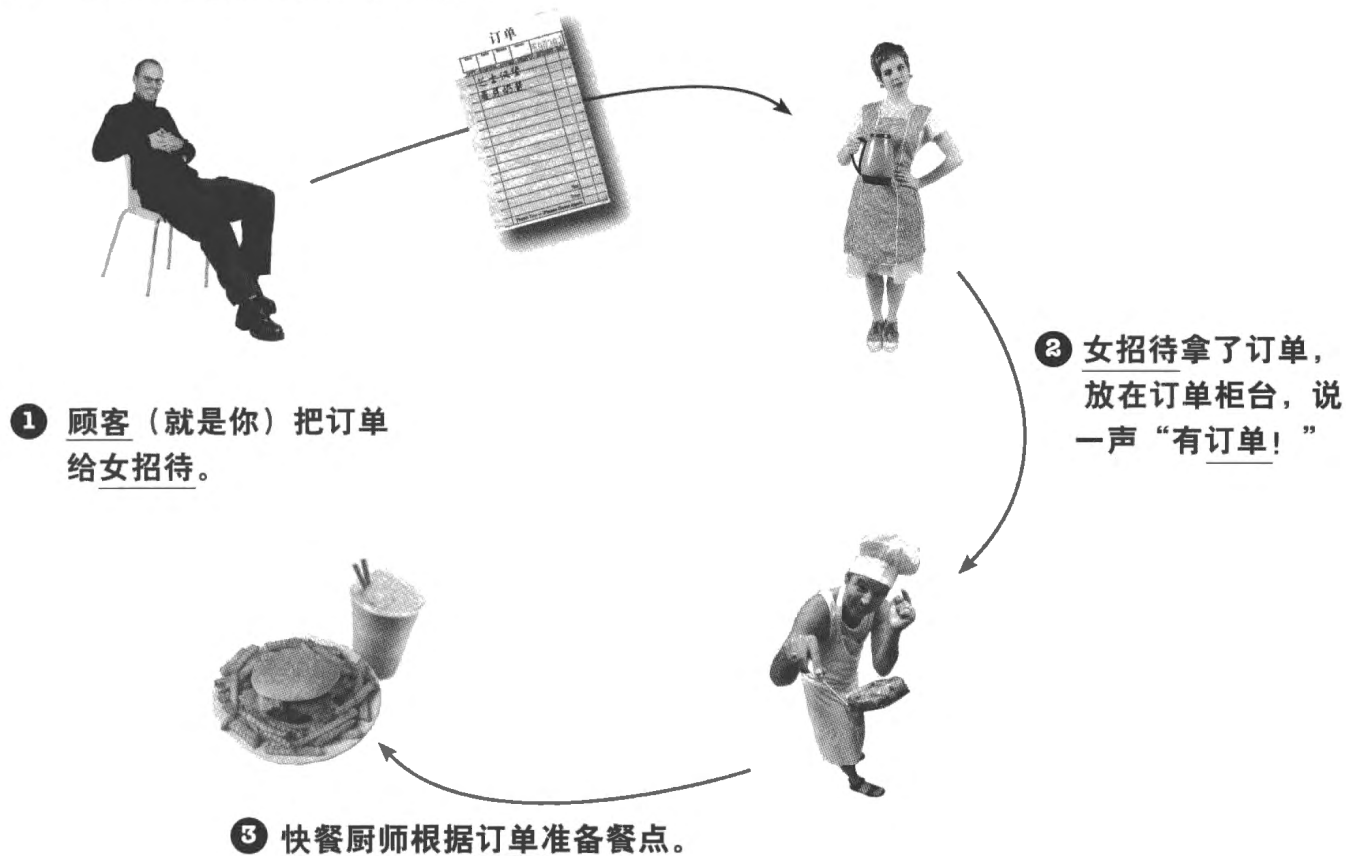
命令模式简介

正如Joe所说，只是通过听一下描述来理解命令模式有点难度。但是别害怕，有一些朋友正准备帮助我们：还记得第1章里出现的友好餐馆吗？距离上次和Alice、Flo以及快餐厨师见面，已经有好一阵子，现在有很好的理由回去（除了食物和很棒的谈话之外）：餐馆可以帮助我们理解命令模式。

因此，我们来绕个路回到餐馆，研究顾客、女招待、订单，以及快餐厨师之间的交互。通过这些交互，你会理解命令模式涉及的对象，也会感受到解耦是如何做到的。之后，我们打算解决遥控器API。

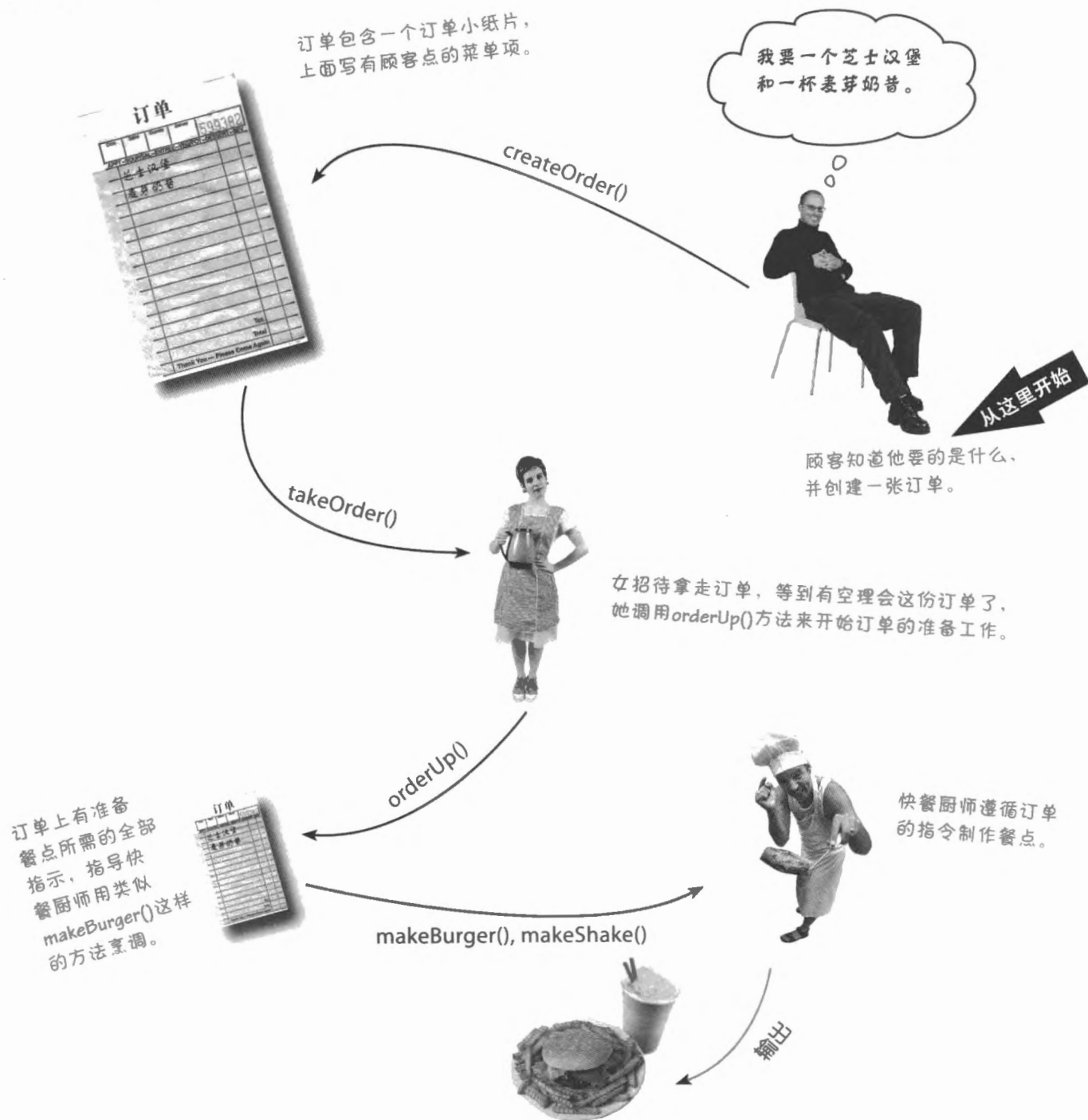
进入对象村餐馆……

OK，我们都知道餐馆是如何运作的：



我们来更详细地研究交互……

……考虑到这家餐馆在对象村，我们也来思考一下涉及的对象和方法调用……



对象村餐馆的角色和责任

一 订单小纸片封装了准备餐点的请求。

把订单小纸片想成一个对象，用来请求准备餐点。和其他对象一样，它可以被四处传递，从女招待到订单柜台，或者到下一班的女招待。它有一个接口，这个接口只包含一个方法，`orderUp()`，该方法封装准备餐点所需的动作。它也有一个到需要准备它的对象（就是快餐厨师）的引用。这些被封装起来，女招待不必知道订单上里有什么，甚至不知道谁准备餐点；她只需要传递订单小纸片到订单窗口，然后喊一声“订单来了”就可以了。

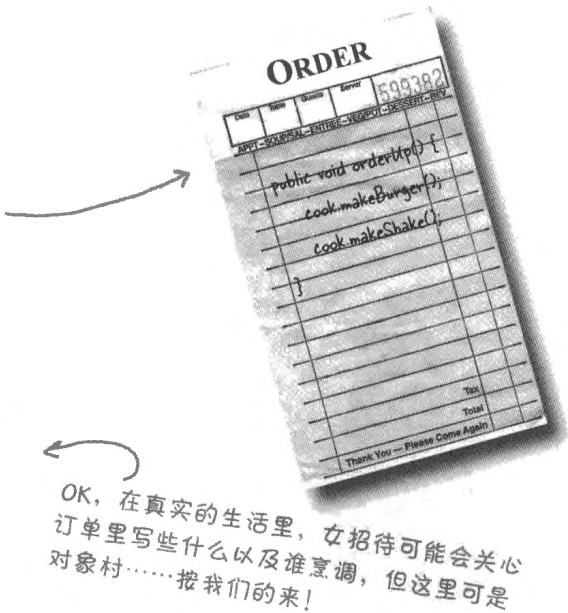
女招待的工作是接受订单，然后调用订单上的 `orderUp()` 方法。

女招待的工作很简单：从顾客那里拿到订单，继续服务各位顾客，直到她带着订单回到柜台。然后，调用`orderUp()`方法让餐点得到准备。正如我们已经讨论的，在对象村中，女招待真的不操心订单上有什么或者谁来准备餐点，她只知道，订单有一个`orderUp()`方法可以调用来完成工作。

现在，一整天，女招待的`takeOrder()`方法得到来自不同顾客的不同订单作为参数，但这不会使她烦心；她知道所有订单都支持`orderUp()`方法。任何时候需要准备餐点，她可以调用`orderUp()`。

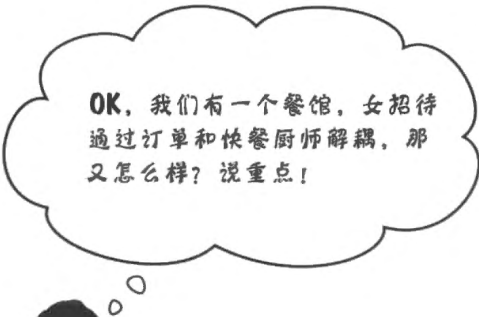
快餐厨师拥有准备餐点所需的知识。

快餐厨师是真正知道如何准备餐点的对象。一旦女招待调用`orderUp()`方法；快餐厨师接管并实现创建餐点所需的所有方法。注意，女招待和厨师之间是彻底解耦的：女招待有封装了餐点细节的订单，她只要调用每个订单上的方法。同样，厨师从订单获得指示，不需要和女招待直接沟通。



OK，在真实的生活里，女招待可能会关心订单里写些什么以及谁烹调，但这里可是对象村……按我们的来！





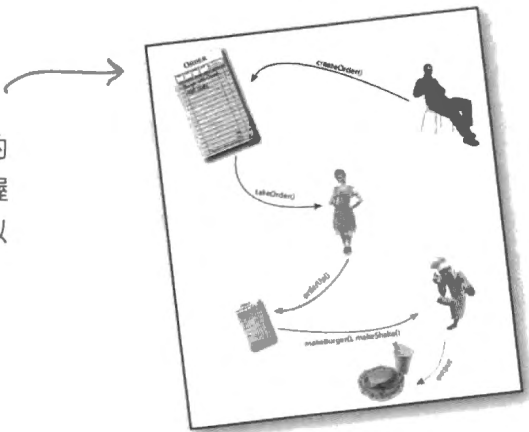
耐心点, 快到重点了……

把餐馆想成OO设计模式的一个模型, 这个模型允许我们把做出请求的对象从接收并执行请求的对象分离。例如, 在我们的遥控器API中, 我们需要把按下按钮时涉及的代码从执行这些请求的厂商特定类的对象分离。要是遥控器的每个槽持有一个像餐馆的订单那样的对象, 会怎么样? 那么, 当一个按钮被按下, 我们可以只调用这个对象上相当于orderUp()的对应方法把灯打开, 遥控器不需要懂得事情如何发生或者什么对象使其发生这些细节。

现在, 我们来把餐间谈话切换成命令模式……



在继续往下走之前, 花一些时间研究两页前的这张图, 图中有餐馆的角色和责任, 务必掌握对象村餐馆的对象和关系。完成之后, 就可以准备将目光集中在命令模式上了!



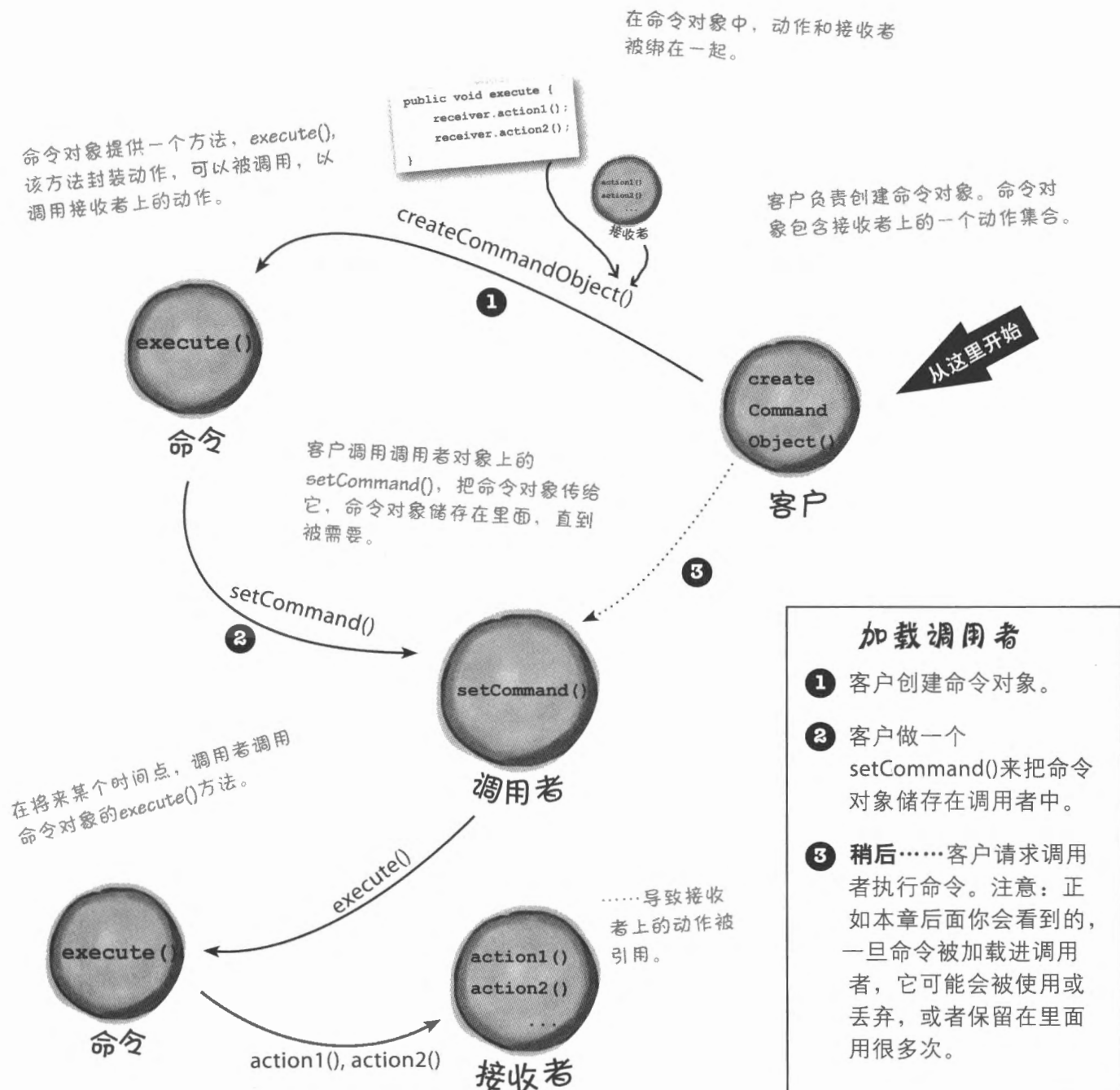
从餐馆到命令模式

OK, 我们在对象村餐馆花了足够多的时间, 我们对所有人员及责任都非常了解。现在我们打算重新绘制餐馆的图, 以反映命令模式。你会看到, 除了名称有变化所有玩家还是一样。

在命令对象中, 动作和接收者被绑定在一起。

命令对象提供一个方法, `execute()`, 该方法封装动作, 可以被调用, 以调用接收者上的动作。

客户负责创建命令对象。命令对象包含接收者上的一个动作集合。



★ ★ ? ★ ★
WHO DOES WHAT?

把餐馆对象和方法匹配到相应的命令模式名称。

餐馆	命令模式
女招待	命令
快餐厨师	<code>execute()</code>
<code>orderUp()</code>	客户
订单	调用者
顾客	接收者
<code>takeOrder()</code>	<code>setCommand()</code>

我们的第一个命令对象

差不多是时候建造我们的第一个命令对象了，对吧？我们来为遥控器编写一些代码。虽然我们还没有弄清楚如何设计遥控器，但自下而上建造一些东西，可能会有帮助……



实现Command接口

要事第一：所有命令对象都实现同一接口，该接口包含一个方法。在餐馆中，我们称这个方法为orderUp()；但是，我们通常使用名称execute()。

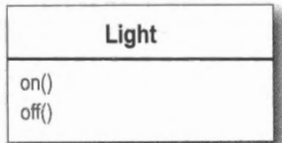
Command接口如下：

```
public interface Command {
    public void execute();
}
```

简单。我们所需的只是一个方法，execute()。

实现一条开灯命令

现在，假设你要实现一条开灯的命令。根据厂商类，Light类有两个方法：on()和off()。你可以实现为一条命令：



```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }
}
```

这是一条命令，因此需要实现Command接口。

构造器被传入命令要控制的特定灯（比如说客厅的灯），并藏在灯实例变量中。当execute被调用时，灯对象成为请求的接收者。

execute()方法调用接收对象上的on()方法，该对象就是我们正在控制的灯。

现在，有了LightOnCommand类，我们来看是否可以用上……

使用命令对象

OK，我们来简化一下：假设我们有一个遥控器，它只有一个按钮以及相应的槽来放置要控制的设备：

```
public class SimpleRemoteControl {
    Command slot;
    public SimpleRemoteControl() {}

    public void setCommand(Command command) {
        slot = command;
    }

    public void buttonWasPressed() {
        slot.execute();
    }
}
```

我们有一个槽持有命令，命令控制一个设备。

我们有一个方法，用于设置控制槽的命令，如果这段代码的客户要改变遥控器按钮的行为，可以多次调用这个方法。

当按钮被按下，这个方法被调用。我们要做的是把当前命令绑定到槽，并调用其execute()方法。

创建一个简单的测试来使用遥控器

下面只有一点点代码，用来测试上面的简单遥控器。我们来看看，并指出代码片段如何匹配命令模式的图：

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        LightOnCommand lightOn = new LightOnCommand(light);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
    }
}
```

用命令模式的说法，这是客户。

遥控器是调用者，会被传入一个可以用来做出请求的命令对象。

现在，我们创建了一个Light对象，也就是请求的接收者。

这里，创建一条命令并把接收者传给它。

这里，把命令传给调用者。

然后，我们模拟按下按钮。

这是运行测试代码的输出。

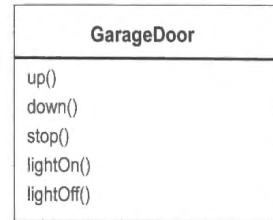
```
File Edit Window Help DinerFoodYum
%java RemoteControlTest
Light is On
%
```



削尖你的铅笔

OK，是时候实现GarageDoorOpenCommand类了。首先，为以下类填充代码。GarageDoor类图在右边。

```
public class GarageDoorOpenCommand
    implements Command {
```



← 你的代码写在这里。

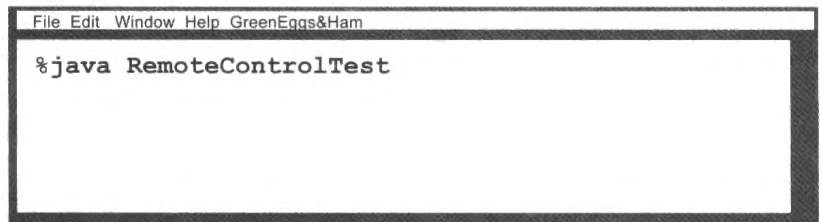
```
}
```

有了类之后，以下代码的输出会是什么？（提示：GarageDoor的up()方法完成时会打印出“Garage Door is Open”）

```
public class RemoteControlTest {
    public static void main(String[] args) {
        SimpleRemoteControl remote = new SimpleRemoteControl();
        Light light = new Light();
        GarageDoor garageDoor = new GarageDoor();
        LightOnCommand lightOn = new LightOnCommand(light);
        GarageDoorOpenCommand garageOpen =
            new GarageDoorOpenCommand(garageDoor);

        remote.setCommand(lightOn);
        remote.buttonWasPressed();
        remote.setCommand(garageOpen);
        remote.buttonWasPressed();
    }
}
```

你的输出在这里。



定义命令模式

经过对象村餐馆的学习，你部分实现了遥控器API，在这个过程中，你也理解了命令模式内的类和对象是如何交互的。现在我们打算定义命令模式并敲定所有细节。

我们从其官方定义开始：

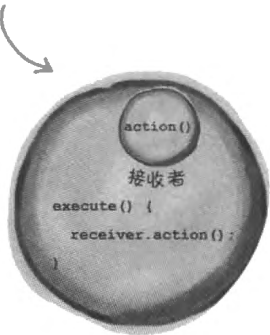
命令模式把请求封装为对象，以使用不同的请求、队列或者日志请求来参数化其他对象，并支持可撤销的操作。

我们来一点点看。我们知道，命令对象通过在特定接收者上绑定一组动作来封装请求。要达到这一点，它把动作和接收者包进一个对象中，该对象只暴露一个方法，execute()。当被调用时，execute()引起接收者上的动作被调用。从外面看，没有其他对象真正知道什么接收者执行了什么动作，只知道如果调用execute()方法，请求的目的就能达到。

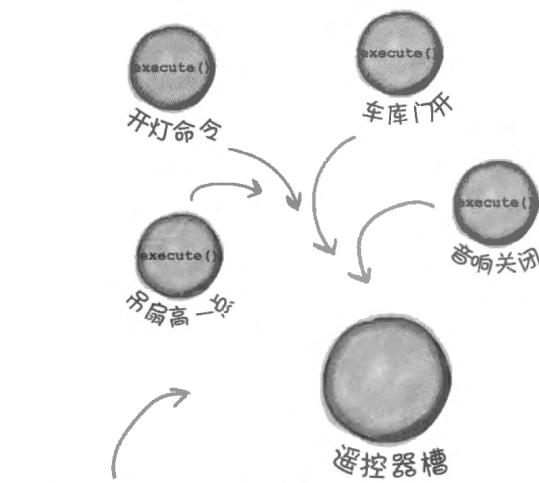
我们也看到了用命令来参数化对象的一些例子。回到餐馆，一天下来，女招待被多个订单参数化。在简单遥控器中，我们首先用一个“开灯”命令加载按钮槽，稍后又用一个“车库门开”命令取代它。像女招待一样，遥控器槽不在乎所拥有的是什么命令对象，只要它实现Command接口就可以了。

我们还没有碰到的是使用命令来实现队列、日志和支持撤销操作。别担心，对这些，基本命令模式有相当直接的扩展，一会我们就会看到。一旦我们有了基本的模式，我们也可以轻易地支持所谓元命令模式（Meta Command Pattern）。元命令模式能让你创建命令的宏，这样，你可以一次执行多个命令。

封装了的请求。

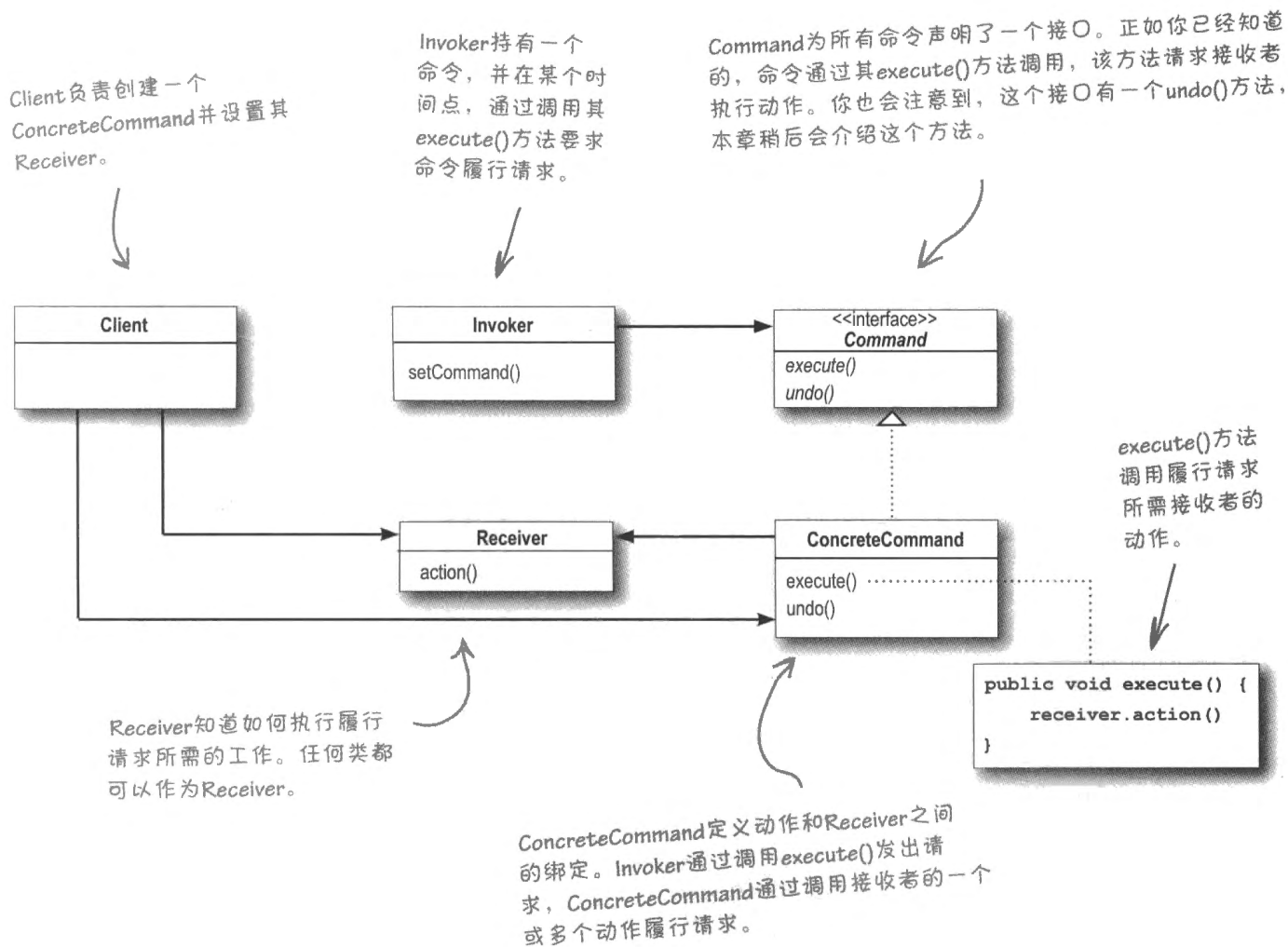


命令



一个调用者——例如遥控器的一个槽——可以用不同的请求参数化。

定义命令模式：类图



命令模式的设计如何支持请求的调用者和接收者之间的解耦？

OK，我想现在我对命令模式很有感觉了。很棒的技术，Joe，我想，在完成这个遥控器API之后，我们会被视为超级巨星。

Mary: 我也这样想。那我们从哪里开始？

Sue: 就像在简单遥控器（SimpleRemote）中所做的，我们需要提供一种分配命令到槽的方式。我们有七个槽，每个槽都有开和关按钮，因此可以像这样分配命令给遥控器：

```
onCommands[0] = onCommand;  
offCommands[0] = offCommand;
```

七个命令槽每个都类似这样。

Mary: 有道理，除了Light对象。遥控器怎么分辨起居室和厨房的灯？

Sue: 啊，是这样——它做不到！遥控器除了在按下按钮时知道如何调用相应命令对象上的execute()之外，不知道其他任何事情。

Mary: 耶，我有点明白了，但是在实现中，我们如何确保正确的对象来打开和关闭正确的设备？

Sue: 当我们创建要加载到遥控器的命令时，我们创建一个LightCommand，绑定到起居室灯对象，另一个LightCommand绑定到厨房灯对象。记住，请求的接收者绑定到封装了它的命令。因此，在按下按钮时，不用在意哪个灯是哪个，当execute()被调用时，正确的事情就发生了。

Mary: 我想我懂了。我们来实现遥控器吧，一切会越来越清楚！

Sue: 听起来很棒，我们来试试看……

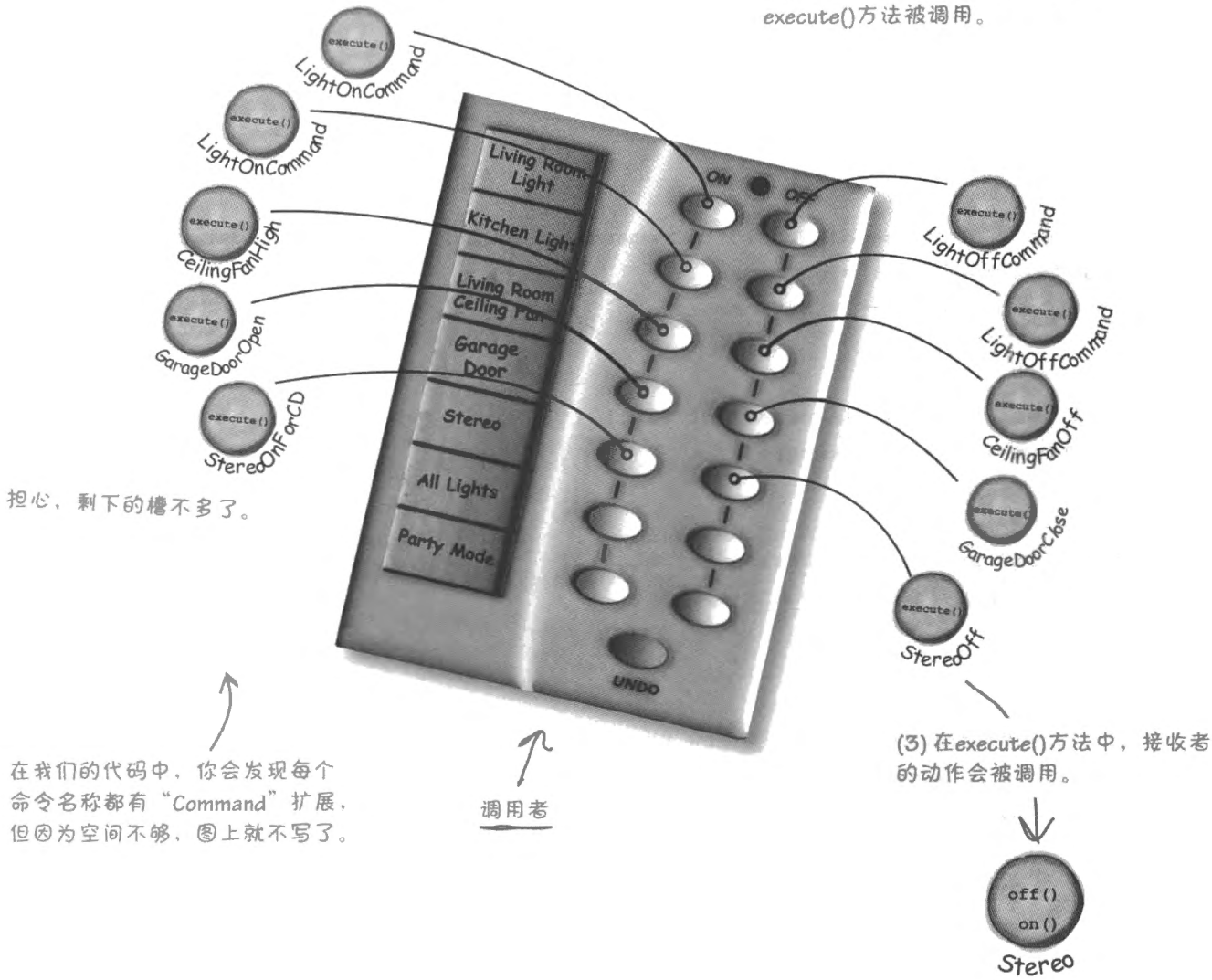


分配命令到槽

因此这样做：我们打算分配命令到遥控器的每个槽，使遥控器成为调用者。当按钮被按下时，相应命令上的execute()方法会被调用，导致接收者(像灯、吊扇和音响)上的动作被调用。

(1) 每个槽有一个命令。

(2) 当按下按钮时，相应命令的execute()方法被调用。



实现遥控器

```
public class RemoteControl {
    Command[] onCommands;
    Command[] offCommands;
```

这个时候，遥控器要处理七个开和关命令，我们把这些命令放在相应数组中。

```
public RemoteControl() {
    onCommands = new Command[7];
    offCommands = new Command[7];
```

在构造器中，我们需要做的是实例化和初始化On和Off数组。

```
    Command noCommand = new NoCommand();
    for (int i = 0; i < 7; i++) {
        onCommands[i] = noCommand;
        offCommands[i] = noCommand;
    }
}
```

setCommand()方法的参数是槽的位置以及储存在槽中的开命令和关命令。

```
public void setCommand(int slot, Command onCommand, Command offCommand) {
    onCommands[slot] = onCommand;
    offCommands[slot] = offCommand;
}
```

它把这些命令放进On和Off数组，以备后面使用。

```
public void onButtonWasPushed(int slot) {
    onCommands[slot].execute();
}
```

```
public void offButtonWasPushed(int slot) {
    offCommands[slot].execute();
}
```

当开或关按钮被按下，硬件负责调用相应的方法，onButtonWasPushed()或offButtonWasPushed()。

```
public String toString() {
    StringBuffer stringBuff = new StringBuffer();
    stringBuff.append("\n----- Remote Control ----- \n");
    for (int i = 0; i < onCommands.length; i++) {
        stringBuff.append("[slot " + i + "] " + onCommands[i].getClass().getName()
            + " " + offCommands[i].getClass().getName() + "\n");
    }
    return stringBuff.toString();
}
```

我们覆盖toString()，以打印出每个槽及其相应的命令。当我们测试遥控器时会用到。

实现命令

嗯，我们已经尝试为SimpleRemoteControl 实现LightOnCommand。我们可以把同样的代码插入到这里，一切都运行得很漂亮。Off命令也没有什么不同；事实上，LightOffCommand 看起来像这样：

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }
}
```

LightOffCommand的运行方式和LightOnCommand一样，除了绑定接收者到不同的动作：off()方法。

我们来尝试一些更有挑战性的事情。为音响编写开和关命令怎么样？OK，关容易，只要把Stereo绑定到StereoOffCommand的off()方法。On有点复杂。假设我们要编写一个StereoOnWithCDCommand……

```
public class StereoOnWithCDCommand implements Command {
    Stereo stereo;

    public StereoOnWithCDCommand(Stereo stereo) {
        this.stereo = stereo;
    }

    public void execute() {
        stereo.on();
        stereo.setCD();
        stereo.setVolume(11);
    }
}
```

就像LightOnCommand一样，我们传入要控制的音响的实例，把它存在一个实例变量中。

要履行这个请求，我们需要调用音响的三个方法：首先，打开，然后设置成播放CD，最后把音量设置为11。为什么是11？嗯，总比10好些，对吧？

Stereo
on()
off()
setCd()
setDvd()
setRadio()
setVolume()

还不错。看看剩下的厂商类，现在你肯定可以搞定剩下的Command 类。

逐步测试遥控器

我们对遥控器做了相当多的工作，剩下需要做的就是运行一些测试以及弄一些描述API的文档。Home Automation or Bust公司肯定对我们的工作印象深刻，你觉得呢？我们打算拿出一个绝佳的设计，让他们能够生产易于维护的遥控器。将来，他们也很容易说服厂商编写一些简单的命令类，因为它们写起来如此简单。

我们来测试这份代码吧！

```
public class RemoteLoader {
```

```
    public static void main(String[] args) {
        RemoteControl remoteControl = new RemoteControl();
```

```
        Light livingRoomLight = new Light("Living Room");
        Light kitchenLight = new Light("Kitchen");
        CeilingFan ceilingFan = new CeilingFan("Living Room");
        GarageDoor garageDoor = new GarageDoor("Garage");
        Stereo stereo = new Stereo("Living Room");
```

} 在合适的位置创建所有设备。

```
        LightOnCommand livingRoomLightOn =
            new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
            new LightOffCommand(livingRoomLight);
        LightOnCommand kitchenLightOn =
            new LightOnCommand(kitchenLight);
        LightOffCommand kitchenLightOff =
            new LightOffCommand(kitchenLight);
```

} 创建所有灯命令对象。

```
        CeilingFanOnCommand ceilingFanOn =
            new CeilingFanOnCommand(ceilingFan);
        CeilingFanOffCommand ceilingFanOff =
            new CeilingFanOffCommand(ceilingFan);
```

} 为吊扇创建开和关。

```
        GarageDoorUpCommand garageDoorUp =
            new GarageDoorUpCommand(garageDoor);
        GarageDoorDownCommand garageDoorDown =
            new GarageDoorDownCommand(garageDoor);
```

} 为车库创建上和下命令。

```
        StereoOnWithCDCommand stereoOnWithCD =
            new StereoOnWithCDCommand(stereo);
        StereoOffCommand stereoOff =
            new StereoOffCommand(stereo);
```

} 创建音响的开和关命令。

```
remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);
remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);
remoteControl.setCommand(2, ceilingFanOn, ceilingFanOff);
remoteControl.setCommand(3, stereoOnWithCD, stereoOff);
```

现在我们有了全部命令，可以把它们加载进遥控槽。

```
System.out.println(remoteControl);
```

```
remoteControl.onButtonWasPushed(0);
remoteControl.offButtonWasPushed(0);
remoteControl.onButtonWasPushed(1);
remoteControl.offButtonWasPushed(1);
remoteControl.onButtonWasPushed(2);
remoteControl.offButtonWasPushed(2);
remoteControl.onButtonWasPushed(3);
remoteControl.offButtonWasPushed(3);
```

在这里，我们用`toString()`方法，打印每个遥控器槽以及分配给它的命令（注意，`toString()`在这里自动被调用，因此我们不必显式调用`toString()`）。

好了，一切准备就绪！
现在，逐步按下每个槽的开和关按钮。

现在，我们来看看遥控器测试的执行……

File Edit Window Help CommandsGetThingsDone

```
% java RemoteLoader
----- Remote Control -----
[slot 0] LightOnCommand          LightOffCommand
[slot 1] LightOnCommand          LightOffCommand
[slot 2] CeilingFanOnCommand     CeilingFanOffCommand
[slot 3] StereoOnWithCDCommand   StereoOffCommand
[slot 4] NoCommand              NoCommand
[slot 5] NoCommand              NoCommand
[slot 6] NoCommand              NoCommand
```

开的槽 关的槽

```
Living Room light is on
Living Room light is off
Kitchen light is on
Kitchen light is off
Living Room ceiling fan is on high
Living Room ceiling fan is off
Living Room stereo is on
Living Room stereo is set for CD input
Living Room stereo volume set to 11
Living Room stereo is off
%
```

我们的命令在行动！记住，每个设备的输出都来自厂商类。例如，当一个灯对象被打开，它会打印“Living Room light is on”。



等一下，槽4到槽6加载的是NoCommand，这是怎么回事？是要忽悠我？

没错，我们确实在这里隐匿了一点东西。在遥控器中，我们不想每次引用槽时都检查命令是否加载。例如，在onButtonWasPushed()方法中，我们需要类似这样的代码：

```
public void onButtonWasPushed(int slot) {
    if (onCommands[slot] != null) {
        onCommands[slot].execute();
    }
}
```

因此，我们怎样绕开上面的做法？实现一个不做事情的命令！

```
public class NoCommand implements Command {
    public void execute() { }
}
```

然后，在RemoteControl构造器中，我们缺省分配每个槽一个NoCommand对象，以便每个槽总是有一些命令可以调用。

```
Command noCommand = new NoCommand();
for (int i = 0; i < 7; i++) {
    onCommands[i] = noCommand;
    offCommands[i] = noCommand;
}
```

因此，在测试运行的输出中，你会看到除了分配命令的槽之外，其他槽分配了缺省的NoCommand对象，在我们创建RemoteControl构造器时分配。



**Pattern
Honorable
Mention**

NoCommand对象是一个空对象（null object）的例子。当你不想返回一个有意义的对象时，以及你要把处理null的责任从客户移除时，空对象就很有用。例如，在我们的遥控器中，没有一个有意义的对象来分配给每个槽，因此我们提供一个NoCommand对象作为替身，在execute()方法被调用时，不做任何事情。

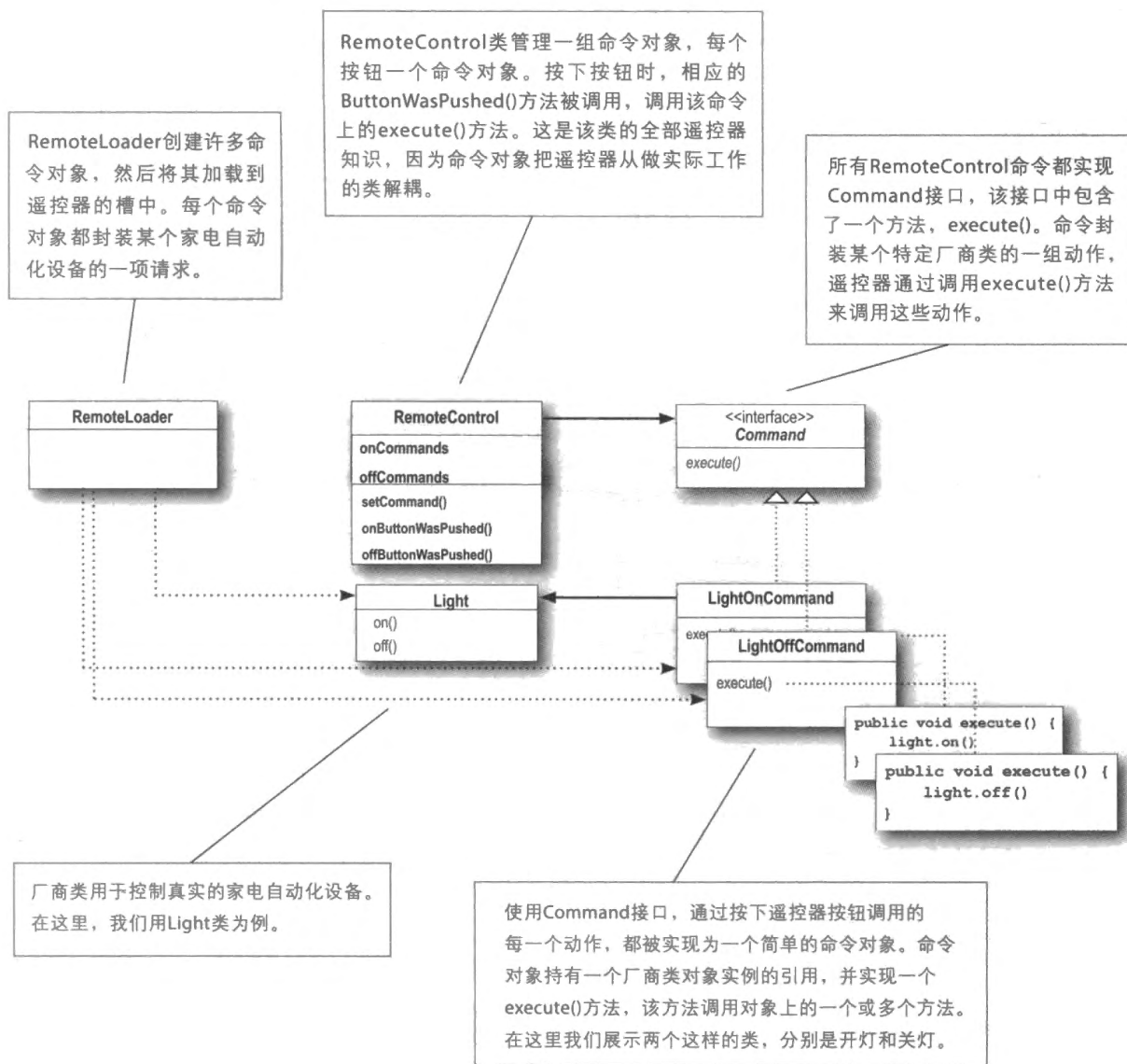
你会发现在许多设计模式相关的内容中用到了空对象，有时你甚至会看到“空对象”出现在设计模式列表中。

是时候编写文档了.....

为Home Automation or Bust公司设计的遥控器API

我们很高兴为您提供以下家庭自动化遥控器的设计和应用编程接口。我们首要的设计目标是让遥控器代码尽可能保持简单，这样一来，当增加新的厂商类时，遥控器不需要修改。因此，我们采用了命令模式，从逻辑上将遥控器类和厂商类解耦。我们相信这将降低遥控器的生产成本，并大大地减少将来的维护成本。

以下类图提供了设计的全貌：





Serious Coding

需要命令模式的代码再上一个台阶吗？你可以使用Java的lambda表达式来跳过创建那些具体命令对象的步骤。用了lambda表达式，不用实例化具体命令对象，而是用函数对象代替。换句话说，我们可以用函数对象作为命令。另外，这样做之后，我们也就删除所有具体命令类。

我们来看看，如何使用lambda表达式作为命令来简化我们之前的代码：

使用lambda表达式更新过的代码如下：

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControl remoteControl = new RemoteControl();

        Light livingRoomLight = new Light("Living Room");
        ...
        LightOnCommand livingRoomLightOn =
        new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
        new LightOffCommand(livingRoomLight);
        ...
        remoteControl.setCommand(0, () -> livingRoomLight.on(),
                                () -> livingRoomLight.off());
        ...
    }
}
```

我们正常创建一个Light对象……

但我们可以移除具体LightOnCommand和LightOffCommand对象。

后面，当你点击按钮之一时，遥控器调用该按钮槽中的命令对象的方法，即lambda表达式。

我们编写具体命令为lambda表达式，做的工作和具体命令的execute()方法相同：也就是，开灯或者关灯。

一旦我们替换具体命令为lambda表达式，我们可以删除所有具体命令类（LightOnCommand、LightOffCommand、HottubOnCommand、HottubOffCommand等）。如果你为每个具体命令如此做了，遥控器应用的类总数就会从22减少到9。

注意，只有在Command接口有一个抽象方法的时候，你可以这样做。一旦添加第二个抽象方法，lambda的简略表达就不行了。

如果你喜欢这个技巧，可以看看你喜爱的Java参考资料来获得更多的lambda表达式信息。

做得好！看来你已经得到了一个了不起的设计，但你是不是忘了顾客要求的某个小东西？像撤销按钮？！



哎呀！差点忘了……幸运的是，一旦我们有了基本的Command类，撤销很容易添加。下面我们来一步步把撤销添加到我们的命令以及遥控器……

我们要做什么？

OK，我们需要添加支持遥控器上撤销按钮的功能。它工作起来像这样，比如，起居室灯是关的，你按下遥控器的打开按钮，显然，灯就开了。现在如果你按下撤销按钮，上一个动作会被倒转，在这个例子中，灯会被关闭。在进入更复杂的例子之前，我们先来让灯用上撤销按钮：

- 1 当命令支持撤销时，就有一个undo()方法和execute()方法形成镜像。不管execute()最后做了什么，undo()倒转过来。因此，在我们可以给命令添加撤销之前，我们需要添加一个undo()方法到Command接口：

```
public interface Command {
    public void execute();
    public void undo();
}
```

新的undo()方法在这里。

够简单的。

现在，我们来深入Light命令，并实现undo()方法。

- ② 我们从LightOnCommand开始：如果LightOnCommand的execute()方法被调用，那么on()方法是最后被调用的。我们知道undo()需要通过调用off()方法做相反的事情。

```
public class LightOnCommand implements Command {
    Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.on();
    }

    public void undo() {
        light.off();
    }
}
```

execute()开灯，因此undo()关灯。

小意思！现在来看LightOffCommand。其undo()方法需要调用Light的on()方法。

```
public class LightOffCommand implements Command {
    Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }

    public void undo() {
        light.on();
    }
}
```

这里，undo()把灯打开。

还有比这更简单的吗？OK，事情还没完。我们需要再做些工作，让遥控器跟踪最后按下的命令按钮和撤销按钮。

- 3 为了添加对撤销按钮的支持，我们不得不对遥控器类做一些小小的改变。我们打算这么做：添加一个新的实例变量来跟踪上一条被调用的命令。然后，无论何时撤销按钮被按下，我们检索此命令并调用其undo()方法。

```
public class RemoteControlWithUndo {
    Command[] onCommands;
    Command[] offCommands;
    Command undoCommand;
```

这是我们为撤销按钮而储藏上一条已执行命令的地方。

```
    public RemoteControlWithUndo() {
        onCommands = new Command[7];
        offCommands = new Command[7];
```

```
        Command noCommand = new NoCommand();
        for(int i=0;i<7;i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
        undoCommand = noCommand;
    }
```

和其他槽一样，撤销开始于一个noCommand，因此在按下其他按钮之前按下撤销，不会做任何事情。

```
    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }
```

```
    public void onButtonWasPushed(int slot) {
        onCommands[slot].execute();
        undoCommand = onCommands[slot];
    }
```

当按钮被按下，我们取得该命令，首先执行它，然后保存它的一个引用在undoCommand实例变量中。针对开和关的命令，做法都一样。

```
    public void offButtonWasPushed(int slot) {
        offCommands[slot].execute();
        undoCommand = offCommands[slot];
    }
```

```
    public void undoButtonWasPushed() {
        undoCommand.undo();
    }
```

当撤销按钮被按下，我们调用存储在undoCommand中的命令的undo()方法，撤销上一条已执行命令的操作。

```
    public String toString() {
        // toString代码在这里……
    }
```

更新，添加undoCommand。

```
}
```

是时候QA 撤销按钮了!

OK, 我们来稍微修改测试程序来测试撤销按钮:

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();

        Light livingRoomLight = new Light("Living Room"); ← 创建一个Light以及新支持undo()
                                                             的开关命令。

        LightOnCommand livingRoomLightOn =
            new LightOnCommand(livingRoomLight);
        LightOffCommand livingRoomLightOff =
            new LightOffCommand(livingRoomLight);

        remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);

        remoteControl.onButtonWasPushed(0);
        remoteControl.offButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
        remoteControl.offButtonWasPushed(0);
        remoteControl.onButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
    }
}
```

把灯的命令添加到遥控器0号槽。

开灯, 然后关灯, 然后撤销。

然后, 关灯, 撤销。

测试结果如下……

```
File Edit Window Help UndoCommandsDefyEntropy
% java RemoteLoader
Light is on ← 开灯, 然后关灯。
Light is off

----- Remote Control -----
[slot 0] LightOnCommand      LightOffCommand
[slot 1] NoCommand          NoCommand
[slot 2] NoCommand          NoCommand
[slot 3] NoCommand          NoCommand
[slot 4] NoCommand          NoCommand
[slot 5] NoCommand          NoCommand
[slot 6] NoCommand          NoCommand
[undo] LightOffCommand

Light is on ← 撤销按钮被按下……LightOffCommand
              的undo()把灯打开。
Light is off
Light is on ← 然后我们又把灯关掉了。

----- Remote Control -----
[slot 0] LightOnCommand      LightOffCommand
[slot 1] NoCommand          NoCommand
[slot 2] NoCommand          NoCommand
[slot 3] NoCommand          NoCommand
[slot 4] NoCommand          NoCommand
[slot 5] NoCommand          NoCommand
[slot 6] NoCommand          NoCommand
[undo] LightOnCommand

Light is off ← 撤销被按下, 因此灯又关上了。
              现在undo持有上一条被激发的命令
              LightOnCommand。
```

Light的命令在这里。

现在undo持有上一条被激发的命令LightOffCommand。

现在undo持有上一条被激发的命令LightOnCommand。

用状态实现撤销

OK，在Light上实现撤销让我们学到了东西，但有点太容易了。通常要实现撤销，需要管理一些状态。我们来尝试一些更有趣的，像来自厂商类的CeilingFan（吊扇）。CeilingFan类允许有多种转速，当然也有一个关闭的方法。

CeilingFan类的源代码如下：

```
public class CeilingFan {
    public static final int HIGH = 3;
    public static final int MEDIUM = 2;
    public static final int LOW = 1;
    public static final int OFF = 0;
    String location;
    int speed;

    public CeilingFan(String location) {
        this.location = location;
        speed = OFF;
    }

    public void high() {
        speed = HIGH;
        // 设置转速为高的代码
    }

    public void medium() {
        speed = MEDIUM;
        // 设置转速为中的代码
    }

    public void low() {
        speed = LOW;
        // 设置转速为低的代码
    }

    public void off() {
        speed = OFF;
        // 关闭吊扇的代码
    }

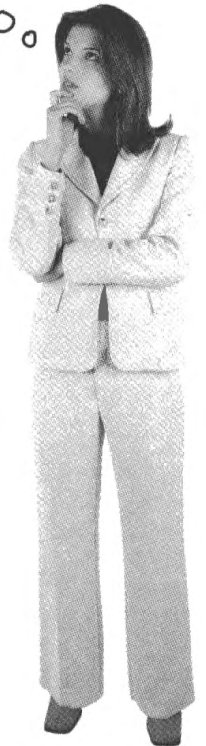
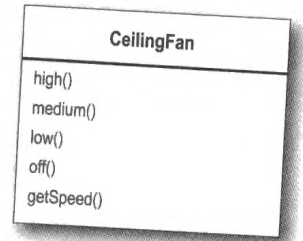
    public int getSpeed() {
        return speed;
    }
}
```

注意，CeilingFan类持有局部状态，代表吊扇的转速。

Hmm，嗯，要恰当地实现
undo，我必须考虑吊扇之前的
转速……

这些方法设置吊扇
的转速。

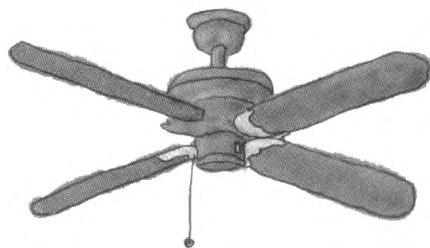
我们可以用getSpeed()
得到吊扇的当前转速。



添加撤销到吊扇命令

现在我们来添加撤销到各种吊扇命令。要做到这一点，我们需要跟踪吊扇最后设置的转速，如果undo()被调用，就重置吊扇到之前的设置。

下面是CeilingFanHighCommand的代码：



```
public class CeilingFanHighCommand implements Command {
    CeilingFan ceilingFan;
    int prevSpeed;
```

我们添加局部状态来跟踪吊扇之前的转速。

```
    public CeilingFanHighCommand(CeilingFan ceilingFan) {
        this.ceilingFan = ceilingFan;
    }
```

```
    public void execute() {
        prevSpeed = ceilingFan.getSpeed();
        ceilingFan.high();
    }
```

在execute()中，改变吊扇的转速之前，需要先记录它之前的状态，以便需要撤销动作时使用。

```
    public void undo() {
        if (prevSpeed == CeilingFan.HIGH) {
            ceilingFan.high();
        } else if (prevSpeed == CeilingFan.MEDIUM) {
            ceilingFan.medium();
        } else if (prevSpeed == CeilingFan.LOW) {
            ceilingFan.low();
        } else if (prevSpeed == CeilingFan.OFF) {
            ceilingFan.off();
        }
    }
}
```

我们把吊扇的转速设置回之前的转速，达到撤销的目的。



我们还有三个吊扇的命令要写：low（低速）、medium（中速）和off（关闭）。你知道如何实现这些命令吗？

准备测试吊扇

是时候把吊扇命令加载到遥控器了。我们打算把吊扇的中速设置加载到0号槽的on按钮，高速设置加载到1号槽。两者相应的 off按钮持有吊扇的关闭命令。

测试脚本如下：

```
public class RemoteLoader {

    public static void main(String[] args) {
        RemoteControlWithUndo remoteControl = new RemoteControlWithUndo();

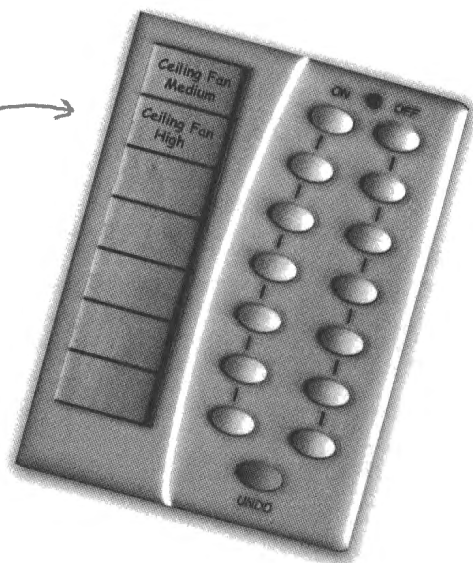
        CeilingFan ceilingFan = new CeilingFan("Living Room");

        CeilingFanMediumCommand ceilingFanMedium =
            new CeilingFanMediumCommand(ceilingFan);
        CeilingFanHighCommand ceilingFanHigh =
            new CeilingFanHighCommand(ceilingFan);
        CeilingFanOffCommand ceilingFanOff =
            new CeilingFanOffCommand(ceilingFan);

        remoteControl.setCommand(0, ceilingFanMedium, ceilingFanOff);
        remoteControl.setCommand(1, ceilingFanHigh, ceilingFanOff);

        remoteControl.onButtonWasPushed(0);
        remoteControl.offButtonWasPushed(0);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();

        remoteControl.onButtonWasPushed(1);
        System.out.println(remoteControl);
        remoteControl.undoButtonWasPushed();
    }
}
```



在这里我们实例化三个命令，高速、中速和关闭。

在这里，我们把中速放进0号槽，高速放进1号槽，并加载这两个槽的关闭命令。

首先，以中速打开吊扇。

然后关闭。

撤销！它应该会回到中速……

这个时候以高速打开。

再做一次撤销，它应该会回到中速。

测试吊扇……

OK，我们来启动遥控器，加载命令，按下一些按钮！

File Edit Window Help UndoThis!

```
% java RemoteLoader
```

```
Living Room ceiling fan is on medium
```

← 以中速打开吊扇，然后将其关闭。

```
Living Room ceiling fan is off
```

```
----- Remote Control -----
```

```
[slot 0] CeilingFanMediumCommand CeilingFanOffCommand
```

```
[slot 1] CeilingFanHighCommand CeilingFanOffCommand
```

```
[slot 2] NoCommand NoCommand
```

```
[slot 3] NoCommand NoCommand
```

```
[slot 4] NoCommand NoCommand
```

```
[slot 5] NoCommand NoCommand
```

```
[slot 6] NoCommand NoCommand
```

```
[undo] CeilingFanOffCommand
```

← 遥控器的命令在这里……

……undo持有上一条执行的命令 CeilingFanOffCommand，以及之前的中速转速。

```
Living Room ceiling fan is on medium
```

← 撤销上一条命令，回到中速。

```
Living Room ceiling fan is on high
```

← 现在，转到高速。

```
----- Remote Control -----
```

```
[slot 0] CeilingFanMediumCommand CeilingFanOffCommand
```

```
[slot 1] CeilingFanHighCommand CeilingFanOffCommand
```

```
[slot 2] NoCommand NoCommand
```

```
[slot 3] NoCommand NoCommand
```

```
[slot 4] NoCommand NoCommand
```

```
[slot 5] NoCommand NoCommand
```

```
[slot 6] NoCommand NoCommand
```

```
[undo] CeilingFanHighCommand
```

← 现在，高速是上一条被执行的命令。

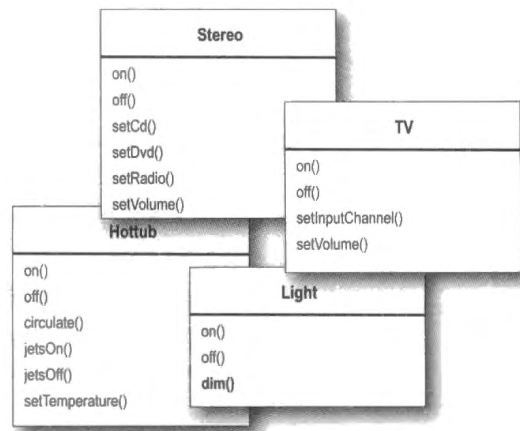
```
Living Room ceiling fan is on medium
```

← 再一次撤销，吊扇回到中速。

```
%
```

每个遥控器都需要“Party模式”！

如果遥控器不能按一个按钮就能同时把灯调暗、打开音响和电视以及打开热水器，那么要这个遥控器还有什么意义？



嗯！我们的遥控器需要为每个设备准备一个按钮，我不认为可以做到。

等一下，Sue，这可不一定。我认为我们可以做到这一点，根本不需要改变遥控器！

Mary的想法是，制造一种新的命令，用来执行一堆其他命令……而不是只执行一个命令！这个想法相当不错，对吧？

```
public class MacroCommand implements Command {
    Command[] commands;
```

```
    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }
```

在宏命令（MacroCommand）中，用命令数组储存一大群命令。

```
    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }
}
```

当宏被遥控器执行时，一次性执行这些命令。

使用宏命令

我们来一步步看如何使用宏命令：

- ❶ 首先，我们创建要放到宏里面的命令集：

```
Light light = new Light("Living Room");
TV tv = new TV("Living Room");
Stereo stereo = new Stereo("Living Room");
Hottub hottub = new Hottub();

LightOnCommand lightOn = new LightOnCommand(light);
StereoOnCommand stereoOn = new StereoOnCommand(stereo);
TVOnCommand tvOn = new TVOnCommand(tv);
HottubOnCommand hottubOn = new HottubOnCommand(hottub);
```

创建所有设备：灯、电视、
音响，以及热水器。

现在，创建所有的On命令
来控制它们。



削尖你的铅笔

我们也需要关闭按钮的命令，请在这里写下创建它们的代码：

- ❷ 接下来，我们创建两个数组，其中一个用来放On命令，另一个用来放Off命令，并把相应命令加载到里面：

```
Command[] partyOn = { lightOn, stereoOn, tvOn, hottubOn};
Command[] partyOff = { lightOff, stereoOff, tvOff, hottubOff};
```

为On命令和Off命令
创建一个数组……

```
MacroCommand partyOnMacro = new MacroCommand(partyOn);
MacroCommand partyOffMacro = new MacroCommand(partyOff);
```

……创建两个相应的宏
来持有它们。

- ❸ 然后，我们像之前一样分配MacroCommand到按钮：

```
remoteControl.setCommand(0, partyOnMacro, partyOffMacro);
```

和其他命令一样，分配宏
命令到按钮。

④ 最后，我们只需要按下某些按钮，看看是否运行正常。

```
System.out.println(remoteControl);
System.out.println("--- Pushing Macro On---");
remoteControl.onButtonWasPushed(0);
System.out.println("--- Pushing Macro Off---");
remoteControl.offButtonWasPushed(0);
```

输出在这里。



File Edit Window Help You Can't Beat A Babka

```
% java RemoteLoader
```

```
----- Remote Control -----
```

这是两个宏命令。



```
[slot 0] MacroCommand      MacroCommand
[slot 1] NoCommand         NoCommand
[slot 2] NoCommand         NoCommand
[slot 3] NoCommand         NoCommand
[slot 4] NoCommand         NoCommand
[slot 5] NoCommand         NoCommand
[slot 6] NoCommand         NoCommand
[undo] NoCommand
```

```
--- Pushing Macro On---
```

```
Light is on
Living Room stereo is on
Living Room TV is on
Living Room TV channel is set for DVD
Hottub is heating to a steaming 104 degrees
Hottub is bubbling!
```

当我们调用“开”宏时，此宏内所有的命令都被执行……



```
--- Pushing Macro Off---
```

```
Light is off
Living Room stereo is off
Living Room TV is off
Hottub is cooling to 98 degrees
```

然后我们调用“关”宏，看起来正常。





我们的MacroCommand唯一缺少的是撤销功能。在宏命令执行后，按下撤销的按钮，那么宏内每一道被激发的命令都必须撤销之前的动作。以下是MacroCommand的代码，请实现undo()方法：

```
public class MacroCommand implements Command {
    Command[] commands;

    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }

    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }

    public void undo() {
        
    }
}
```

没有蠢问题

问：一定需要接收者吗？为什么不能让命令对象实现execute()方法的细节？

答：一般来说，我们尽量争取制作“哑”命令对象，它只调用接收者上的一个动作；但是，有很多“聪明”命令对象的例子，它们实现大多数（如果不是全部的话）履行请求所需的逻辑。无疑，你可以这样做，只是要牢记，调用者和接收者之间不再有同样级别的解耦，你也不能用接收者来参数化命令。

问：我怎样才能实现撤销操作的历史？换句话说，我要能够多次按下撤销按钮。

答：很棒的问题。其实相当容易，不只是保持一个到上一个已执行命令的引用，而是保持一个之前命令的堆栈。然后，无论何时按下撤销，调用者把第一项弹出栈，并调用其undo()方法。

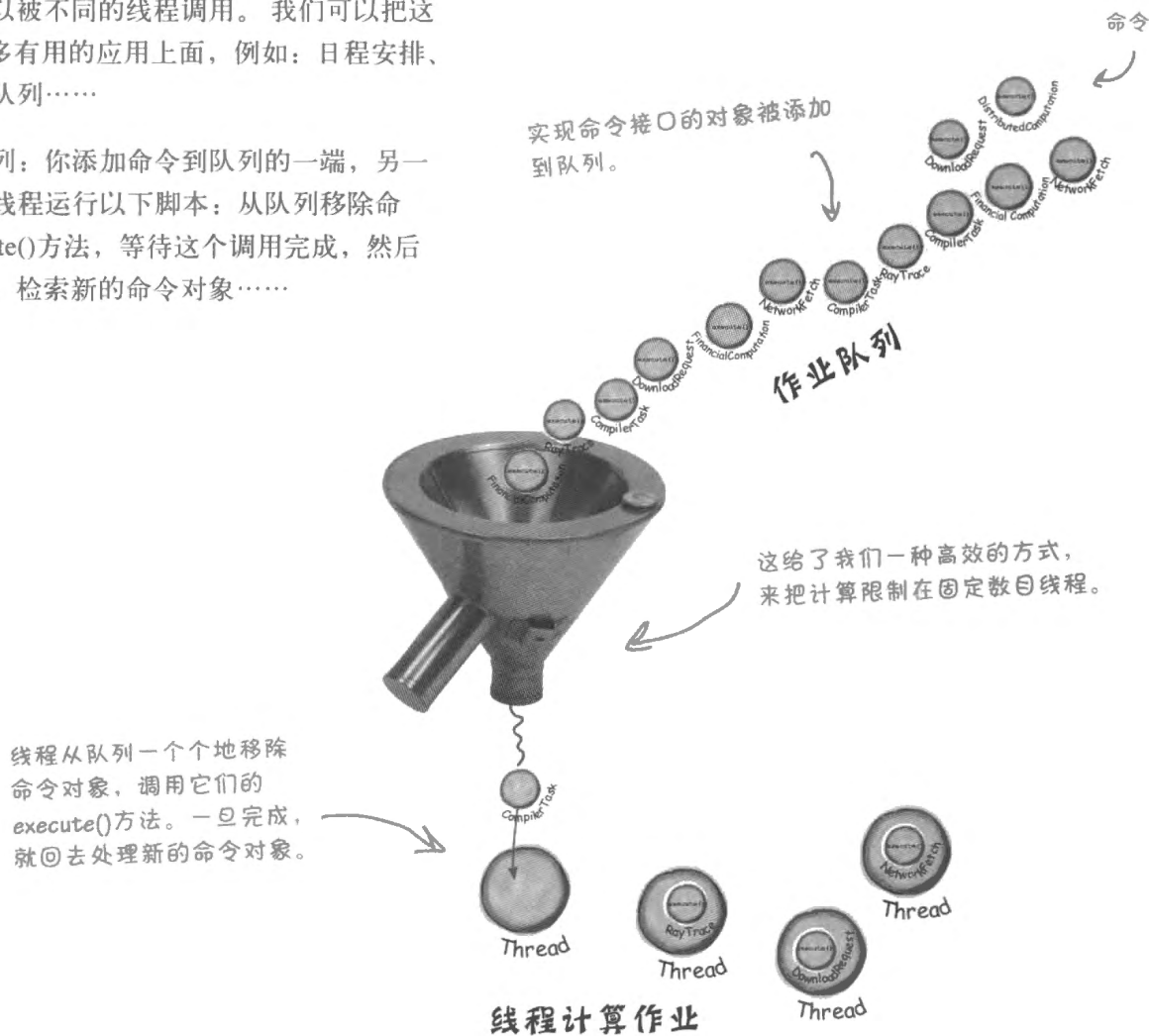
问：通过创建一个PartyCommand，并把执行其他命令的调用放进PartyCommand的execute()方法中，我把party模式实现为一个命令，这样可以吗？

答：可以，但是本质上这是“硬编码”party模式到PartyCommand，干吗要自找麻烦呢？有了宏命令，你可以动态地决定哪一个命令进入宏命令，因此，使用宏命令有更多弹性。一般来说，宏命令是更加优雅的方案，代码更少。

命令模式的更多用途：请求队列

命令把一小块计算打包（一个接收者和一组动作），然后把它作为头等对象传来传去。即使在一些客户应用创建命令对象很久之后，计算自身依然可能被调用。事实上，甚至可以由不同的线程调用。我们可以把这个场景应用到许多有用的应用上面，例如：日程安排、线程池以及作业队列……

想象一个作业队列：你添加命令到队列的一端，另一端有一组线程。线程运行以下脚本：从队列移除命令，调用其execute()方法，等待这个调用完成，然后丢弃该命令对象，检索新的命令对象……



注意，作业队列类完全从做计算的对象解耦。这一分钟，线程可能做财务计算，下一分钟可能从网络检索信息。作业队列对象不关心，它们只检索命令并调用execute()。同样，只要你把对象放进实现命令模式的队列，当线程可用时，execute()方法就会被调用。



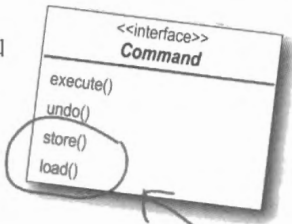
Web服务器如何使用这样的队列？
你还能想到任何其他的应用吗？

命令模式的更多用途：日志请求

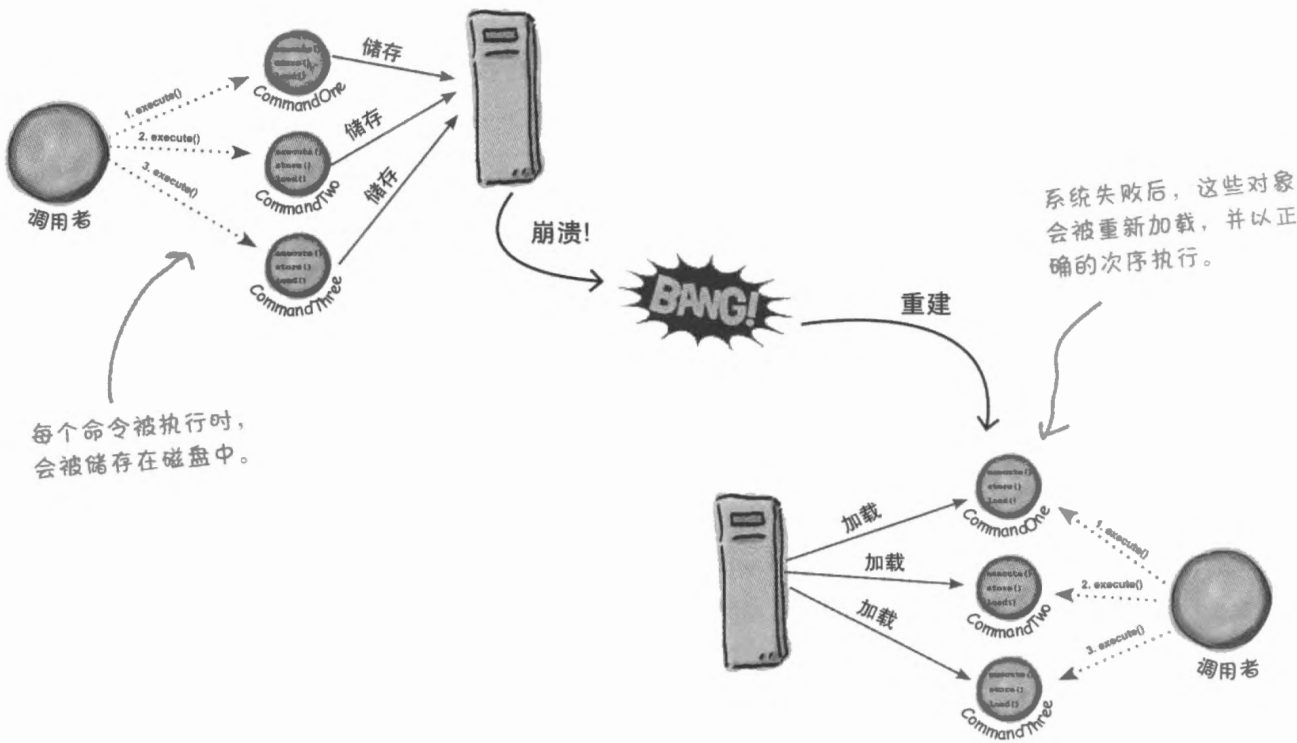
某些应用需要我们把所有动作都记录在日志中，并能在系统崩溃之后，通过重新调用这些动作来恢复。通过添加两个方法store()和load()，命令模式能够支持这一点。在Java中，我们可以使用对象序列化来实现这些方法，但是，通常的告诫是序列化最好只用在持久化上。

那怎么做呢？当我们执行命令时，把历史储存在磁盘中。一旦出现崩溃，就重新加载命令对象，并成批地依次调用它们的execute()方法。

这种记录日志的方式对遥控器来说没有意义，但是，有很多应用调用大型数据结构的动作，不能每一次出现变化都快速保存。通过记录日志，我们可以保存上次检查点之后的所有操作，如果系统出状况，把这些操作应用到检查点。例如，电子表格应用：我们可能要通过记录动作日志实现失败恢复，而不是每一次发生改变时写一个电子表格的拷贝。在更多高阶应用中，这些技巧可以被扩展应用到事务方式的操作集，所有操作都必须完成，否则一个都不做。



我们为记录日志添加两个方法。



现实世界中的命令模式

还记得第2章的小小生活改变应用吗？

那一章中，我们看到Java的Swing库中到处是观察者，以动作监听者的形式监听（或观察）用户界面组件上的事件。

动作监听者不只是一个观察者接口，也是一个命令接口，我们的AngelListener和DevilListener类不只是观察者，也是具体命令。没错，一个例子里有两个模式！



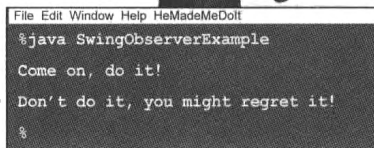
这是我们可爱的接口。

这是当我们点击按钮时的输出。



恶魔的回答 →

天使的回答 →



削尖你的铅笔

以下是第2章小小生活改变应用的代码(重要的片段)。看看你是否能识别谁是客户，谁是命令，谁是调用者以及谁是接收者？

```
public class SwingObserverExample {
    // 设置……
    JButton button = new JButton("Should I do it?");
    button.addActionListener(new AngelListener());
    button.addActionListener(new DevilListener());
    // 在这里设置框架属性
}

class AngelListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Don't do it, you might regret it!");
    }
}

class DevilListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Come on, do it!");
    }
}
}
```



削尖你的铅笔 答案

以下是第2章小小生活改变应用的代码(重要的片段)。看看你是否能识别谁是客户,谁是命令,谁是调用者以及谁是接收者?

答案见下。

```
public class SwingObserverExample {
    // 设置……
```

按钮是调用者。当点击按钮时,按钮调用命令(ActionListener)中的actionPerformed(类似于execute())方法。

```
        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());
```

客户是设置Swing组件和调用者中的命令(AngelListener和DevilListener)的类。

```
        // 在这里设置框架属性
```

```
    }
```

```
class AngelListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Don't do it, you might regret it!");
```

```
    }
```

```
}
```

ActionListener是命令接口:它有一个方法actionPerformed(),该方法像execute()一样,当命令被调用时执行。

```
class DevilListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Come on, do it!");
```

```
    }
```

```
}
```

AngelListener和DevilListener是具体命令。它们实现命令接口(本例即为ActionListener)。

本例的接收者是System对象。记住,调用一个命令导致接收者上的动作。在一个典型的Swing应用中,这会导致调用UI中其他组件上的动作。



你的设计工具箱中的工具

你的工具箱开始变重了！在本章，我们添加了一个模式，这个模式允许我们把方法封装进命令对象，随心所欲地储存、传递和调用它们。

OO基础

OO原则

封装变化。

组合优于继承。

针对接口编程，不针对实现编程。

努力达到交互对象之间的松耦合设计。

类应该对扩展开放，对修改关闭。

依赖抽象，不要依赖具体类。

抽象

封装

多态

继承

OO Patterns

Singleton - Ensure a class only has one instance.

命令，封装请求为对象，这样可以让你用不同的请求、队列或者日志请求来参数化客户，并支持撤销操作。

当你需要把做出请求的对象从知道如何执行请求的对象解耦时，使用命令模式。

要点

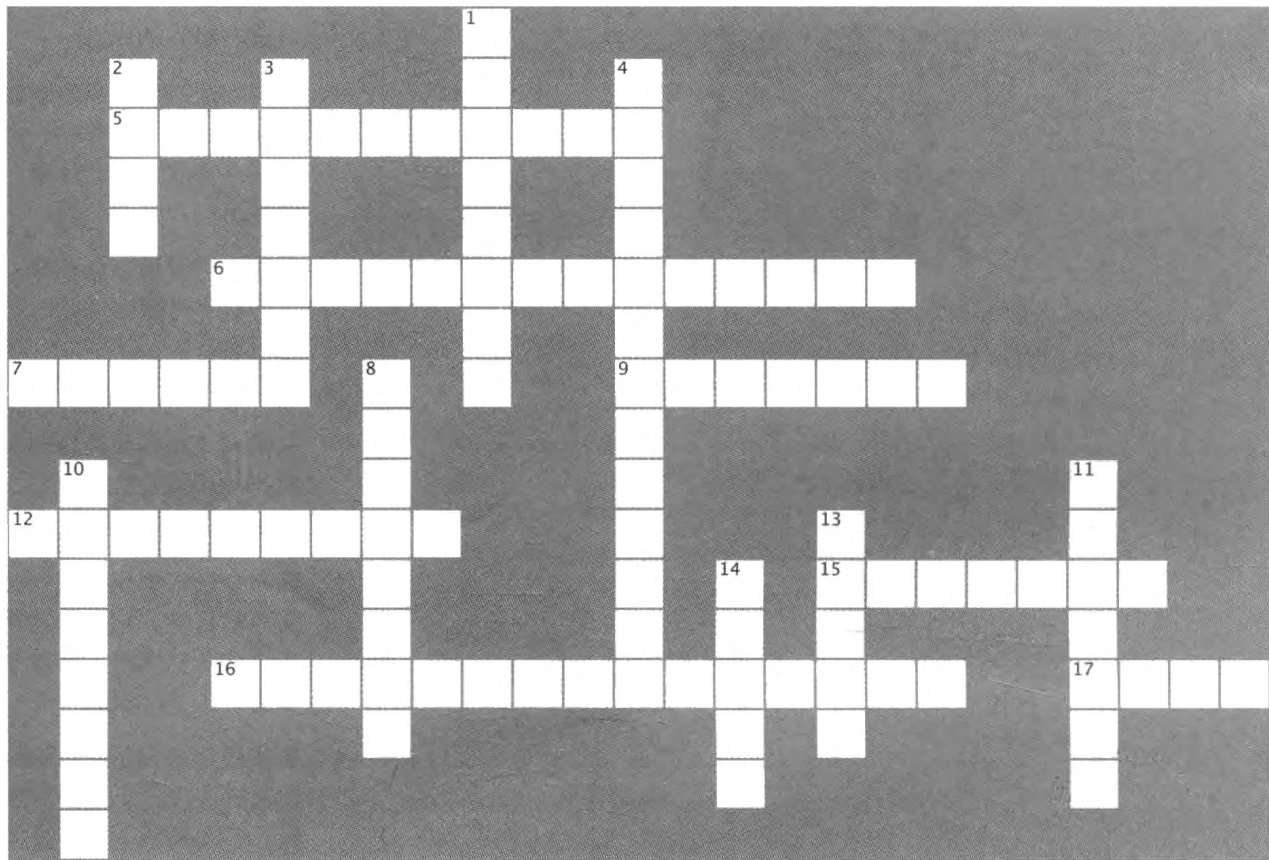
- 命令模式把做出请求的对象从知道如何执行请求的对象解耦。
- 命令对象处在解耦的中心，封装接收者以及一个（或一组）动作。
- 调用者通过调用命令对象的execute()做出请求，这会使得接收者的动作被调用。
- 调用者可以用命令参数化，甚至可以在运行时动态地进行。
- 通过实现一个undo()方法来把对象重建到最后一次执行execute()前的状态，命令可以支持撤销。
- 宏命令是命令模式的一种简单的延伸。它允许调用多个命令。同样，宏命令很容易支持undo()。
- 在实践中，“聪明”命令对象并不少见。这些对象自己实现请求，而不是委托给接收者。
- 命令也可以用来实现日志和事务系统。



设计模式填字游戏

是时候休息一下，平复你的心情了。

这是另一个填字游戏，答案都来自本章的英文词汇。



横排

- 5. 我们喜爱的城市。
- 6. 给我们口头业务的公司。
- 7. 命令模式中顾客的角色。
- 9. 知道动作和接收者的对象。
- 12. 调用者和接收者是_____。
- 15. 女招待是一个_____。
- 16. Dr. Seuss 餐馆的食品（四个词）。
- 17. 命令可以做的另一件事。

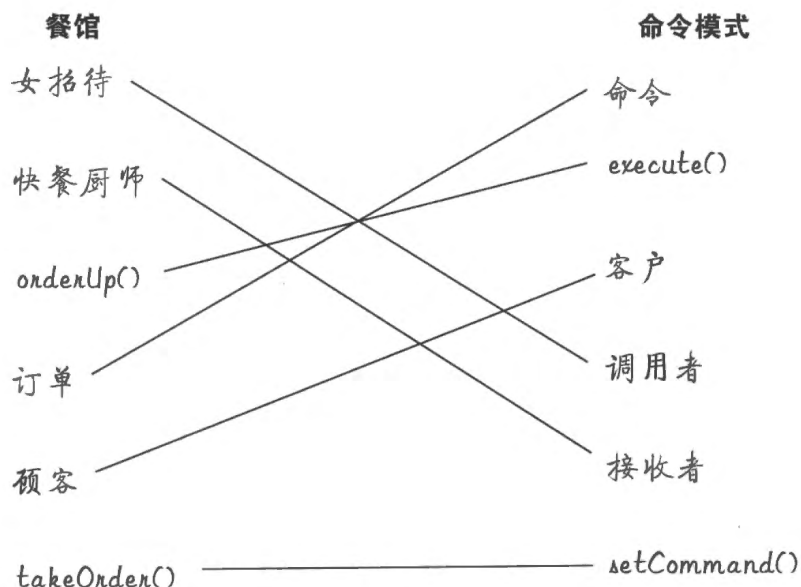
竖排

- 1. 厨师和这个人肯定解耦。
- 2. 女招待不做这个。
- 3. 命令封装这个。
- 4. 作为遥控器中的接收者(两个词)。
- 8. 知道如何把事情搞定的对象。
- 10. 贯彻一个请求。
- 11. 所有命令都提供这个。
- 13. 我们的第一个命令对象控制这个。
- 14. 一个命令_____一个动作集和一个接收者。

WHO DOES WHAT?

答案

把餐馆对象和方法匹配到相应的命令模式名称。



削尖你的铅笔

答案

以下是GarageDoorOpenCommand类的代码。

```
public class GarageDoorOpenCommand implements Command {
    GarageDoor garageDoor;

    public GarageDoorOpenCommand(GarageDoor garageDoor) {
        this.garageDoor = garageDoor;
    }
    public void execute() {
        garageDoor.up();
    }
}
```

输出如下：

```
File Edit Window Help GreenEggs&Ham
%java RemoteControlTest

Light is on
Garage Door is Open
%
```



练习答案

以下是
MacroCommand的
undo()方法。

```
public class MacroCommand implements Command {
    Command[] commands;
    public MacroCommand(Command[] commands) {
        this.commands = commands;
    }
    public void execute() {
        for (int i = 0; i < commands.length; i++) {
            commands[i].execute();
        }
    }
    public void undo() {
        for (int i = commands.length - 1; i >= 0; i--) {
            commands[i].undo();
        }
    }
}
```



削尖你的铅笔 答案

以下是为关闭按钮创建命令的代码。

```
LightOffCommand lightOff = new LightOffCommand(light);
StereoOffCommand stereoOff = new StereoOffCommand(stereo);
TVOffCommand tvOff = new TVOffCommand(tv);
HottubOffCommand hottubOff = new HottubOffCommand(hottub);
```

